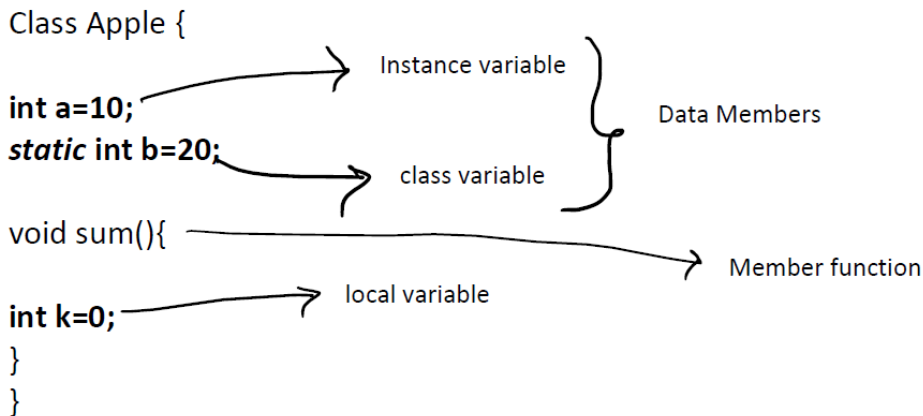


Functions: Memory Occupations and Related Concepts

Types of variables: Instance Variables, Class Variables, Local Variables



**All variables at class level are called Data Members (both static and non static)*

**All functions at the class level are called member functions (both static and non static)*

Instance Variables: These are variables declared within the class, at the class level, without the '**static**' keyword. As the name suggests, instance variables belong to an instance or object of the class. Each time an object is instantiated (created), a copy of these variables is created in that object's memory. This implies that if we have an instance variable '**int a**' and create **N** number of objects, we essentially have **N** instances of this variable '**int a**' spread across the **N** objects.

Class Variables: These are variables declared within the class, at the class level, using the '**static**' keyword. Unlike instance variables, only one set of these variables is stored in memory. Class variables are essentially global variables, shared by all instances of the class.

Local Variables: Variables declared inside functions are referred to as local variables.

Syntax for creating object: Classname object_name = new Constructor(...)

The **new keyword in Java is used to dynamically allocate memory and create an instance of a class, also known as an object*

**The non-static members of a class belong to the object of the class*

**The static members belong to the class itself*

```
class Apple {  
    int a, b, c;  
    static int x, y, z;  
  
    void sum() {  
        int s1 = a + b + c;  
        int s2 = x + y + z;  
        System.out.print(s1);  
        System.out.print(s2);  
    }  
}
```

In the function **sum()**, which is a non-static function, it can access the non-static variables 'a', 'b' and 'c'. Additionally, static members can be accessed from anywhere, so we can also access 'x', 'y' and 'z' since they are static members.

```

static void product() {
    int p1 = x * y * z;
    System.out.println(p1);
}

public static void main(String args[])
{
    x = 5;
    y = 2;
    z = 10;

    product(); //output: 100

    Apple ob1 = new Apple();
    ob1.a = 2;
    ob1.b = 3;
    ob1.c = 4;

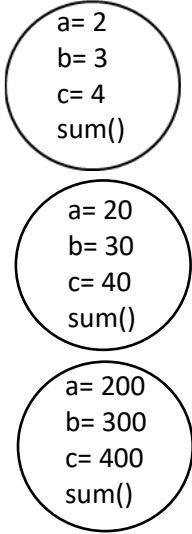
    Apple ob2 = new Apple();
    ob2.a = 20;
    ob2.b = 30;
    ob2.c = 40;

    Apple ob3 = new Apple();
    ob3.a = 200;
    ob3.b = 300;
    ob3.c = 400;

    ob1.sum();    // output: 9   17
    ob2.sum();    // output: 90  17
    ob3.sum();    // output: 900 17
}
}

```

Here the function **product()** is a static function. We can only access static data members from static function **directly**, that is why only 'x', 'y' and 'z' can be accessed here directly. (Although we can still access non static members from static function with a reference to object of the same class)

Class Memory / MetaSpace	Heap Memory
x y z	
product()	
main()	

When we write,
Apple *obj* = new Apple();
we are creating an object of class Apple in heap memory which will contain all non-static members of the class, and the reference of this object will be stored in obj. Here, obj is a reference variable to the object we just created, we can use obj to access this object

***Remember: obj is not an object, it's a reference to the object in memory. it stores the reference of the object**

Calling and accessing rules:

1. We can **directly** access non static members from non static functions
2. We can **directly** access static members from anywhere (static and non-static functions)
3. We can't access non-static members **directly** from static functions, as non static or instance variables belong to object of the class. We must use an object of the class to access non static or instance members from a static function

***** Remember, We can also access static members using object in java, however it's considered as a bad practice, can be confusing, hence it's not recommended**

Variable Shadowing in the Context of Class-Level and Local Variables

Variable shadowing occurs when a **local variable (declared inside a method)** has the **same name** as a **class-level variable (instance or static variable)**. The local variable **hides** the class-level variable within its scope.

Local variables take precedence over class-level variables in the same method.

Use **this.variableName** to access instance variables inside methods.

Use **ClassName.variableName** to access static variables inside methods.

```
class Apple {
    int x = 10;        // Instance variable
    static int y=10;   // Class variable

    void display() {
        int x = 20;    // Local variable shadows instance variable
        int y = 20;    // Local variable shadows class variable
        System.out.println(x);           // Prints 20
        System.out.println(y);           // Prints 20
        System.out.println(this.x);      // Prints 10
        System.out.println(Apple.y);     // Prints 10
    }

    public static void main(String[] args) {
        Apple ob = new Apple();
        ob.display();
    }
}
```

Why is this.x needed?

x = 20 in the display() method hides the class level instance variable x = 10.
this.x explicitly refers to the instance variable (int x = 10 at class level).

Why is Apple.y used?

y = 20 in display() shadows static int y = 10.
Apple.y ensures we access the class variable and not the local one.